A New Multisignature Scheme based on Strong Diffie-Hellman Assumption

Duc-Phong Le and Alban Gabillon

*D-P. Le *and A. Gabillon are with CsySEC/LIUPPA, Universite de Pau, IUT de Mont-de-Marsan, Departement GTR. 371 rue du Ruisseau, BP 201, F-40004 Mont-de-Marsan Cedex, France.*
Email: Duc-Phong.Le@etud.univ-pau.fr; Alban.Gabillon@univ-pau.fr

Digital multisignatures improve security of digital signatures and reduce fraud by collusion. In this paper we first present a new secure signature scheme which is based on a variant of *Strong Diffie-Hellman Assumption*, called SDH2 assumption. Then, we extend this signature for a new multisignature scheme without interaction between users. Our proposed schemes which work in groups equipped with efficient bilinear map are provably secure without random oracles. We also compare our schemes with previous works.

Keywords: Digital signature, Multisignature, SDH assumption, Bilinear map, Without random oracles

1 Introduction

Digital signatures are one of the most important cryptographic primitives. Digital signature schemes have been developed for different applications. Recently, many pairing-based signature schemes have been proposed. Boneh, Lynn and Shacham [BLS01] first presented a Weil pairing-based short signature which is provably secure in the random oracle model. This was followed by a large number of other pairing based signature schemes [ZK02, Bol04, BGLS03, ZSS04]. Pairing-based signature schemes provide shorter signature than traditional public key signature like RSA and DSA with the same level of security. In [BB04a], Boneh and Boyen also proposed a new short signature scheme that is strongly existentially unforgeable under an adaptive chosen message attack in the standard security model (i.e. without the random oracle model). Their signature scheme is based on the SDH assumption.

Multisignature schemes allow any subgroup of a group of users to jointly sign a message such that a verifier is convinced that each member of the subgroup participated in signing. The goal of multisignature is to prove that each member of the stated subgroup signed the message, the size of this subgroup being arbitrary. It is up to a particular application to decide which subgroup is required to sign a message. A verifier might reject a multisignature not because it is invalid, but because the verifier is not satisfied with the subgroup which signed the message. Multisignature can be applied to provide efficient batch verification of several signatures of the same message under different public keys.

The first multisignature scheme was introduced by Itakura and Nakamura in [IN83], and has been followed by many other research works [Oka88, Boy89, OO91]. Those initial schemes were not very efficient and in particular there was no formal notion of security. Micali et al. [MOR01] first formalized the strong notion of security for a variant of multisignature scheme. Their scheme is called *Accountable-Subgroup Multisignature (ASM)* which requires flexibility of subgroups of users (no restriction on the size of subgroups) and accountability of individual users (without use of a trusted third party). Later, Boldyreva [Bol04] presented the first pairing-based multisignature scheme in which multisignature generation does not require signer interaction. Her scheme is based on BLS signature [BLS01] and its security is proven by

*Supported by Conseil General des Landes

applying a variant of the *ASM* model. However, her scheme uses an expensive special hash function called *Map-to-point* that encodes arbitrary finite strings to elements of group $\mathbb{G}$, this function is probabilistic and generally inefficient. Moreover, that scheme is based on *Gap Diffie-Hellman (GDH)* problem in the random oracle model and thus, there is only an heuristic argument for its security.

The notion of *Strong Diffie-Hellman (SDH) Assumption* was first proposed by Boneh and Boyen in [BB04a]. This assumption (or its variants) have been used to construct a number of new cryptosystems, including efficient digital signature [ZSS04], identity based encryption scheme without random oracle [BB04b], blind signatures without random oracle [Oka06]. In this paper, we first present a new signature scheme based on a variant of *SDH assumption*, called *SDH2 assumption*. We also prove that the *SDH2 assumption* and the *SDH assumption* are equivalent. We then extend the proposed signature for a new non-interactive multisignature scheme. Unlike previous schemes, our multisignature scheme is provably secure without random oracles.

The rest of this paper is organized as follows. In section 2, we first introduce some basic knowledge regarding secure signature schemes, bilinear pairings, the *SDH assumption* and the *SDH2 assumption*. Then, we present a new secure signature scheme based on *SDH2 assumption* in section 3. We also prove that this scheme is secure without random oracle on the SDH2 assumption. Then, we propose our multisignature scheme and discuss its security in the section 4. Finally, we conclude this paper in section 5.

2 Preliminaries

In this section, we first present the notion of security of signature schemes. Next, we briefly review some background on groups with efficiently computable bilinear maps. Then, we recall the definition of *Strong Diffie-Hellman (SDH) Assumption* which is introduced in [BB04a], and we finally present a variant of *SDH* assumption, called *SDH2* assumption, on which our constructions are based.

2.1 Signature schemes and their security

Signature schemes. A signature scheme $\mathcal{S}$ is made up of three algorithms, $\mathcal{S} = \{\mathcal{K}, \mathcal{S}, \mathcal{V}\}$, for generating keys, signing, and verifying signatures, respectively. These algorithms work as follows:

- $\mathcal{K}$. The randomized key generation algorithm $\mathcal{K}$ takes the global information I and outputs a pair PK, SK of public and private keys.
- $\mathcal{S}$. The signing algorithm $\mathcal{S}$ takes a private key SK, the global info I and a message M, and returns a signature σ.
- $\mathcal{V}$. The verification algorithm $\mathcal{V}$ takes a public key PK and a signed message (M, σ), and returns valid or invalid.

Security. We now introduce two notions of security for signature schemes: *existential unforgeability* presented in [GMR88] and *strong existential unforgeability* presented in [ADR02] under an adaptive chosen message attack. To define them, we introduce the following game between a challenger $\mathcal{S}$ and an adversary $\mathcal{A}$:

Setup: The challenger $\mathcal{S}$ runs algorithm $\mathcal{K}$ to obtain a public key PK and a private key SK. The adversary $\mathcal{A}$ is given PK.

Queries: The adversary $\mathcal{A}$ adaptively requests signatures on at most q_S messages of its choice $M_1, \ldots, M_{q_s} \in \{0,1\}^*$, under PK. The challenger $\mathcal{S}$ responds to each query with a signature $\sigma_i \leftarrow \mathcal{S}(SK, M_i)$.

Output: Eventually, the adversary $\mathcal{A}$ outputs a pair (M, σ) and wins the game if:

1. M is not any of $M_1, \ldots, M_{q_s}$; and
2. Verify (PK, M, σ) = valid.

We define $\mathsf{Sig\ Adv}_{\mathcal{A}}$ to be the probability that the adversary $\mathcal{A}$ wins in the above game, taken over the coin tosses made by $\mathcal{A}$ and the challenger $\mathcal{S}$.

Definition 2.1 *(Existential unforgeability) Adversary $\mathcal{A}$ (t, q_S, ϵ)-forges a signature scheme if $\mathcal{A}$ runs in time at most t, $\mathcal{A}$ makes at most q_S signature queries, and $\mathsf{Sig\ Adv}_{\mathcal{A}}$ is at least ϵ. A signature scheme is (t, q_S, ϵ)- existentially unforgeable under an adaptive chosen message attack if there exists no forger that (t, q_S, ϵ)-breaks it.*

A digital signature system is said to be secure if it is existentially unforgeable under a chosen message attack. In other words, this mean that an adversary cannot produce a signature for a new message from a signature for a few previously signed messages.

Strong existential unforgeability: If we replace the condition M is not any of $M_1, \ldots, M_{q_s}$ with (M, σ) is not any of $(M_1, \sigma_1), \ldots, (M_{q_s}, \sigma_{q_s})$ then we obtain a stronger notion of security for signature schemes. It is said that the scheme is (t, q_S, ϵ)-strongly existentially unforgeable under an adaptive chosen message attack (for more details see in [ADR02]). For some applications, *strong existential unforgeability* is needed. It ensures that the adversary cannot even produce a new signature for a previously signed message.

2.2 Bilinear Groups

Bilinear map is an efficient tool to implement a decision procedure for the *Diffie-Hellman problem* which is required in our proposed schemes. In this section, we briefly review the necessary facts about bilinear maps and bilinear map groups (for more details, see [DBS04, GPS06]). Consider the following settings:

- $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are three cyclic groups of prime order p;
- g_1 is an arbitrary generator of $\mathbb{G}_1$, g_2 is an arbitrary generator of $\mathbb{G}_2$;
- A map $e : \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T$ is called a bilinear pairing if it satisfies the following properties:
  1. *Bilinearity*: $\forall u \in \mathbb{G}_1, \forall v \in \mathbb{G}_2$ and $\forall a, b \in \mathbb{Z}$, $e\left(u^a, v^b\right) = e\left(u, v\right)^{ab}$;
  2. *Non-degeneracy*: $e\left(g_1, g_2\right) \neq 1$, and is a generator of $\mathbb{G}_T$;
  3. *Computable*: there exists a polynomial algorithm to compute $e\left(u, v\right)$ for $\forall u \in \mathbb{G}_1, \forall v \in \mathbb{G}_2$

In this paper, we consider the general case where $\mathbb{G}_1 \neq \mathbb{G}_2$. This allows us to take advantage of certain families of algebraic curves in order to obtain the shortest possible signatures. Specifically, elements of $\mathbb{G}_1$ have a short representation over the ground field $\mathbb{F}_q$ whereas elements of $\mathbb{G}_2$, which may be defined over an extension field $\mathbb{F}_{q^\alpha}$, have a longer representation than those of $\mathbb{G}_1$. In the more simple case $\mathbb{G}_1 = \mathbb{G}_2$, such a bilinear map, also called symmetric pairing, is found on supersingular curves [BLS01, BF01]. This type of curves takes a number of advantages. In particular, they are implemented very efficiently in characteristic three for computing the Tate pairing [BGHS07]. However, those curves also pose questions of their long term security. In fact, there are very few usable curves with the correct properties. Moreover there also exists many efficient algorithms for the discrete logarithm problem in fields of small characteristic [PSV06].

All group operations and the bilinear map must be efficiently computable. Joux and Nguyen [JN03] showed that an efficiently computable bilinear map provides an algorithm for solving the *Decision Diffie-Hellman (DDH)* problem when $\mathbb{G}_1 = \mathbb{G}_2$. In [BB00] Boneh and Boyen proved this to be still true in the more general case where $\mathbb{G}_1 \neq \mathbb{G}_2$. They defined the general notion of bilinear group as follow.

Definition 2.2 ***(Boneh and Boyen, [BB00])*** *We say that $(\mathbb{G}_1, \mathbb{G}_2)$ is a bilinear group pair if there exists a group $\mathbb{G}_T$ and a non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \mapsto \mathbb{G}_T$, such that the group order $p = |\mathbb{G}_1| = |\mathbb{G}_2| = |\mathbb{G}_T|$ is prime, with the pairing e, the group operations in $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ being all efficiently computable.*

It is easy to show that the DDH problem can be efficiently solved when $\mathbb{G}_1 \neq \mathbb{G}_2$ by using the pairing to test whether $e(g_1^a, g_2) = e(g_1, g_2^b)$ in $\mathbb{G}_T$.

The signature scheme proposed by Boneh and Boyen [BB04a] and several other cryptosystems, which use a bilinear map where $\mathbb{G}_1 \neq \mathbb{G}_2$, assume the existence of an efficient computable homomorphism

$\psi : \mathbb{G}_2 \mapsto \mathbb{G}_1$. The assumption of the existence of such homomorphisms is usually required to get proofs of security of schemes (see [GPS06, SV07] for a discussion more details). In analysis of Galbraith et al. [GPS06], such a bilinear map is called Type 2. The curves of Type 2 are ordinary and the homomorphism ψ from $\mathbb{G}_2$ to $\mathbb{G}_1$ is the trace map. In this paper, we will not need an isomorphism from $\mathbb{G}_2$ to $\mathbb{G}_1$. Hence, our schemes can use any bilinear group construction currently known, in particular, our schemes can use Type 3 groups [GPS06] which offer good performance and flexibility for high security parameters.

Earlier bilinear pairings, namely Weil pairing and Tate pairing of algebraic curves were used in cryptography for the MOV attack [MOV93] (Weil pairing) and FR attack [FR94] (Tate pairing). These attacks reduce the discrete logarithm problem on some elliptic or hyperelliptic curves to the discrete logarithm problem in a finite field. Modified Weil Pairing [BF01] and Tate Pairing [BLS04, BGHS07] are examples of cryptographic bilinear maps. The later pairing is, in practice, much more efficiently computable than the former. On algebraic curves in particular, such pairings are very efficiently computable using Miller's algorithm [Mil04]. Currently, active research is being carried out to obtain efficient algorithms to compute pairings.

2.3 Strong Diffie-Hellman (SDH) Assumption

SDH problem was first introduced by Boneh and Boyen in [BB04a, BB04b]. Let $\mathbb{G}_1$ and $\mathbb{G}_2$ be two cyclic groups of prime order p, respectively generated by g_1 and g_2. *q-Strong Diffie-Hellman (q-SDH)* problem is stated as follows:

Given as input a $(q+2)$-tuple of elements $\left(g_1, g_2, g_2^x, g_2^{x^2} \ldots, g_2^{x^q}\right) \in \mathbb{G}_1 \times \mathbb{G}_2^{q+1}$, a homomorphism $\psi(g_2) = g_1$, and output a pair $(c, g_1^{1/(x+c)}) \in \mathbb{Z}_p \times \mathbb{G}_1$ for a freely chosen value $c \in \mathbb{Z}_p \backslash \{-x\}$,

an algorithm $\mathcal{A}$ has advantage ϵ in solving $q-$SDH in bilinear group pair $(\mathbb{G}_1, \mathbb{G}_2)$ if

$$\mathsf{SDH\ Adv_{q,\mathcal{A}}} := Pr\left[\mathcal{A}\left(g_1, g_2, g_2^x, \ldots, g_2^{x^q}\right) = \left(c, g_1^{\frac{1}{x+c}}\right)\right] \geq \epsilon$$

where the probability $\mathsf{SDH\ Adv_{q,\mathcal{A}}}$ is over the random choice of generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$, the random choice of $x \in \mathbb{Z}_p^{\times}$, and the random bits consumed by $\mathcal{A}$.

Definition 2.3 *We say that the $(q, t, \epsilon)-$SDH assumption holds in $(\mathbb{G}_1, \mathbb{G}_2)$ if no $t-$time algorithm has advantage at least in solving the $q-$SDH problem in $(\mathbb{G}_1, \mathbb{G}_2)$.*

2.4 Variant of the SDH Assumption

Our scheme is based on a variant of q-SDH problem called q-SDH2 problem which is defined as follows: Given a $(q+3)$-tuple $(g_1, g_1^x, \ldots, , g_1^{x_q}, g_2, g_2^x) \in \mathbb{G}_1^{q+1} \times \mathbb{G}_2^2$ as input, output a triple $(\eta, f, c) \in \mathbb{G}_1 \times \mathbb{Z}_p^2$, such that $\eta^{x+c} = g_1^f$, where $f, c \in \mathbb{Z}_p$.

An algorithm $\mathcal{A}$ has advantage ϵ in solving $q-$SDH2 in bilinear group pair $(\mathbb{G}_1, \mathbb{G}_2)$ if

$$\mathsf{SDH2\ Adv_{q,\mathcal{A}}} := Pr\left[\mathcal{A}\left(g_1, g_1^x, \ldots, , g_1^{x_q}, g_2, g_2^x\right) = (\eta, f, c)\right] \geq \epsilon,$$

where the probability $\mathsf{SDH2\ Adv_{q,\mathcal{A}}}$ is over the random choice of generators $g_1 \in G1$ and $g_2 \in \mathbb{G}_2$, the random choice of $x \in \mathbb{Z}_p^{\times}$, and the random bits consumed by $\mathcal{A}$.

Definition 2.4 *We say that the $(q, t, \epsilon)-$SDH2 assumption holds in $(\mathbb{G}_1, \mathbb{G}_2)$ if no $t-$time algorithm has advantage at least in solving the $q-$SDH2 problem in $(\mathbb{G}_1, \mathbb{G}_2)$.*

Now, we consider the relation between the SDH and SDH2 assumptions.

Theorem 2.1 *The q-SDH2 assumption and the q-SDH assumption are equivalent.*

Proof This theorem can easily be proven as follows. First, we construct a variant of the SDH2 problem called SDH' problem by simplifying SDH2 problem as follows: Given a $(q+3)$-tuple $(g_1, g_1^x, g_1^{x_2}, \ldots, g_1^{x_q}, g_2, g_2^x) \in \mathbb{G}_1^{q+1} \times \mathbb{G}_2^2$ as input, output a pair $(\eta, c) \in \mathbb{G}_1 \times \mathbb{Z}_p$, such that $\eta^{x+c} = g_1$. Then, we indicate that the q-SDH' assumption and the q-SDH assumption are equivalent. This was dealed by Victor Wei

in [Wei05b]. He proved that q-SDH implies the q-SDH' on the assumption that there exists an efficiently homomorphism ψ from $\mathbb{G}_2$ to $\mathbb{G}_1$, $\psi(g_2) = g_1$. Hence, the q-SDH2 assumption and the q-SDH assumption are equivalent. □

Recently, many cryptographic schemes including encryption, signature, and key management schemes were proposed on the basis of the *Strong Diffie-Hellman (SDH)* problem [BB04b, BB04a], or its variants [MSK02, ZSS04, Oka06]. Some tight equivalence reductions among *SDH assumptions* are also given by Wei in [Wei05a]. Cheon in [Che06] also investigated the security analysis of the SDH problem. He showed that SDH and related problems are slightly easier than DLP (*Discrete Logarithm Problem*). However, his analysis is generic and does not violate the generic lower bound on the hardness of SDH assumption given by Boneh and Boyen.

3 Proposed Signature Scheme

In this section, we first propose a new secure signature scheme in standard model under the SDH2 assumption. Then, we discuss its features and its security.

3.1 Signature scheme

Let $(\mathbb{G}_1, \mathbb{G}_2)$ be a bilinear group pair as shown in Section 2.2 where $|\mathbb{G}_1| = |\mathbb{G}_2| = p$ for some prime p, and $m \in \mathbb{Z}_p$. In our description, messages m are elements in $\mathbb{Z}_p$. However, in practice we can sign messages of arbitrary length $\{0,1\}^*$ by using a collision resistant hash function $H_k : \{0,1\}^* \to \{0,1\}^k$. The proposed signature scheme is called $\mathcal{S}$; it consists of three algorithms $\mathcal{S} = (\mathcal{K}, \mathcal{S}, \mathcal{V})$ which behave as follows:

- $\mathcal{S}.\mathcal{K}$. (Key generation phase): Select random generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$, and random integers $x, y \in \mathbb{Z}_p^\times$. Compute $u = g_2^x \in \mathbb{G}_2$, $z = e(g_1, g_2) \in \mathbb{G}_T$ and $v = z^y \in \mathbb{G}_T$. The public key is the tuple (g_1, g_2, u, v, z). The secret key is the pair (x, y).
- $\mathcal{S}.\mathcal{S}$. (Signing phase): Given a secret key (x, y) and a message $m \in \mathbb{Z}_p$, pick randomly $r \in \mathbb{Z}_p \setminus \{-my\}$, $s \in \mathbb{Z}_p \setminus \{-x\}$ and compute $\sigma = g_1^{\frac{r+my}{x+s}} \in \mathbb{G}_1$. The signature is the triple (σ, r, s). Here $\frac{r+my}{x+s}$ is computed modulo p.
- $\mathcal{S}.\mathcal{V}$. (Verification phase): Given a public key (g_1, g_2, u, v, z), a message m, and a signature (σ, r, s), verify that $(\sigma, u, g_2^s, z^r, v^m)$ is a DDH (*Decisional Diffie-Hellman*) tuple by testing whether

$$e(\sigma, u \cdot g_2^s) = z^r \cdot v^m. \tag{1}$$

  If the equality holds then the signature is declared valid; otherwise it is declared invalid.

3.2 Correctness of the scheme

Theorem 3.1 *If Eq.* (1) *holds, then the signature of message* m *is verified.*

Proof We need to show that for all key pairs and all messages, any signature generated by the signing procedure verifies as valid under the corresponding public key. Indeed, we have

$$\begin{aligned} e(\sigma, u \cdot g_2^s) &= e(g_1^{\frac{r+my}{x+s}}, g_2^x \cdot g_2^s) = e(g_1^{\frac{r+my}{x+s}}, g_2^{x+s}) = e(g_1, g_2)^{(x+s)\cdot\frac{r+my}{x+s}} \\ &= e(g_1, g_2)^r \cdot e(g_1, g_2)^{my} = z^r \cdot v^m. \end{aligned}$$

Theorem 3.1 is proven □

3.3 Main features

In this section, we analyze the performance of our scheme, then compare to the Boneh-Boyen scheme [BB04a] which is based on SDH assumption and to Okamoto scheme [Oka06] which is also based on a variant of SDH assumption. We analyze the efficiency of signature schemes in relation to three indicators: signature size, complexity (key generation cost, signing cost, verification cost) and security.

Signature size. A signature contains three elements (σ, r, s), each of length approximately $log_2 p$ bits, therefore the total signature length is approximately $3log_2 p$. It is comparable to that of Okamoto scheme [Oka06] and 1.5 times longer than that of Boneh-Boyen scheme [BB04a]. For example, if we use an elliptic curve described in [BLS01], then we obtain a signature whose length is approximately 510 bits, which is still less than that of RSA based signatures with the same security.

Complexity. Key generation and signature generation times of our scheme are comparable to that of Boneh-Boyen scheme [BB04a]. In the signature generation phase, our scheme uses one scalar multiplication on group $\mathbb{G}_1$ instead of three scalar multiplication in [BB04a]. Hence, our signature generation time is much faster than that of Okamoto [Oka06]. Verification time of our scheme is comparable to that of Boneh-Boyen scheme and faster than that of Okamoto. In this phase, our scheme uses one pairing operation and one multi-scalar multiplication on group $\mathbb{G}_2$ instead of two pairing and three multi-scalar multiplications on group $\mathbb{G}_2$ in Okamoto scheme.

Security. Our scheme is proven secure without the random oracle model. The following theorem 3.2 states that our scheme is strong existentially unforgeable against adaptive chosen message attacks, provided that the SDH2 assumption introduced in Section 2.4 holds in $\mathbb{G}_1, \mathbb{G}_2$. We consider an adversary who makes up to q_S adaptive signature queries, and reduces the forgery to the resolution of a random q-SDH2 instance for $q = q_S$.

Relation to Chameleon Hash Signatures. Similarly to the discussion in [BB04a], the $r+my$ component in our proposed scheme provides us with the functionality of a Chameleon hash: given m, we can choose r so that $r + my$ maps to some predefined value of our choice. This makes possible to handle the chosen message attack. Embedding the hash $r+my$ directly in the signature scheme results is a much more efficient construction than using an explicit chameleon hash (that requires additional exponentiations). Therefore, our new signature scheme is a chameleon signature scheme.

3.4 Security

Theorem 3.2 *Assume the (q, t', ϵ')-SDH2 assumption holds in $(\mathbb{G}_1, \mathbb{G}_2)$. Then, the proposed signature scheme is (t, q_s, ϵ)-secure against strong existential forgery under an adaptive chosen message attack provided that*

$$q = q_s + 1, \quad \epsilon \geq 3q_s\epsilon' \quad \textit{and} \quad t \leq t' - \Theta(q_s^2 T),$$

where T is the maximum time for an exponentiation in $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{Z}_p$.

Proof Assume that $\mathcal{A}$ is a forger that (t, q_s, ϵ)-breaks the signature scheme. We construct an algorithm $\mathcal{B}$ that, by interacting with the forger $\mathcal{A}$, solves the q-SDH2 problem in time t' with advantage ϵ'.

Before describing the algorithm $\mathcal{B}$ we distinguish between three types of forgers that $\mathcal{A}$ can emulate (these types are distinguished in a manner similar to those in [Oka06]). Let (g_1, g_2, u, v, z) be the public key given to $\mathcal{A}$, where $u = g_2^x, z = e(g_1, g_2), v = z^y$. First, we note that by adding as many dummy queries as necessary, we may always assume that $\mathcal{A}$ makes exactly q_s signature queries. Suppose then that $\mathcal{A}$ adaptively asks for signatures on messages $m_1, \ldots, m_{q_s} \in \mathbb{Z}_p$ and is given signatures (σ_i, r_i, s_i) for $i = 1, \ldots, q_s$ in response. Let $b_i = r_i + m_i y$ for each i, and denote by $(m_*, \sigma_*, r_*, s_*) = (m_*, g_1^{(r_*+m_*y)/(x+s_*)}, r_*, s_*)$ the forgery eventually produced by $\mathcal{A}$.

The three types of forgers, $\mathcal{A}$, are as follows:

1. **Type-1 forger** We say that $\mathcal{A}$ is a type-1 forger, denoted $\mathcal{A}_1$, if it outputs forged signature $(m_*, \sigma_*, r_*, s_*)$ such that $s_* \notin \{s_1, s_2, \ldots s_{q_s}\}$.

2. **Type-2 forger** We say that $\mathcal{A}$ is a type-2 forger, denoted $\mathcal{A}_2$, if it outputs forged signature $(m_*, \sigma_*, r_*, s_*)$ such that $s_* \in \{s_1, s_2, \ldots s_{q_s}\}$ (i.e., $s_* = s_k$ for $k \in \{1, \ldots, q_s\}$) and $r_* + m_* y \neq r_k + m_k y$ (mod p).

3. **Type-3 forger** We say that $\mathcal{A}$ is a type-3 forger, denoted $\mathcal{A}_3$, if it outputs forged signature $(m_*, \sigma_*, r_*, s_*)$ such that $s_* \in \{s_1, s_2, \ldots s_{q_s}\}$ (i.e., $s_* = s_k$ for $k \in \{1, \ldots, q_s\}$) and $r_* + m_* y \equiv r_k + m_k y$ (mod p). Note that in this case $r_* \neq r_k$, since $r_* = r_k$ implies $m_* = m_k$ and $\sigma_* = \sigma_k$.

For each type forger $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$, we show how to construct a suitable simulator $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ as follows:

1. **Input:** Algorithm $\mathcal{B}$ is given a random instance $(A_0, A_1, \ldots, , A_q, g_2, h)$ of the q-SDH2 problem in $(\mathbb{G}_1, \mathbb{G}_2)$, where $A_i = g_1^{x^i}$ for $i = 0, \ldots, q$, and $h = g_2^x \in \mathbb{G}_2$, for some unknown $x \in \mathbb{Z}_p$.
2. **Coin flip:** Algorithm $\mathcal{B}$ first picks a random value $c_{type} \in \{1, 2, 3\}$ that indicates its guess for the type of forger that $\mathcal{A}$ will emulate. The subsequent actions performed by $\mathcal{B}$ differ with $c_{type} \in \{1, 2, 3\}$ as follows:
3. **Type-1 forger.** ($c_{type} = 1$) First, we describe the simulator $\mathcal{B}_1$, which interacts with a type-1 forger $\mathcal{A}_1$ to produce a forged signature as follows:
   (a) **Key setup:** Algorithm $\mathcal{B}_1$ randomly selects $y, s_i (i = 1, \ldots, q-1) \in \mathbb{Z}_p$.
   Let $f(X)$ be a polynomial of variable X such that $f(X) \leftarrow \prod_{i=1}^{q-1}(X + s_i) \bmod p$. Expand $f(X)$ and write $f(X) = \sum_{i=0}^{q-1} \alpha_i X^i$, where $\alpha_0, \alpha_1, \ldots, \alpha_{q-1} \in \mathbb{Z}_p$ are the coefficients of the polynomial $f(X)$. Clearly, algorithm $\mathcal{B}_1$ can efficiently calculate $\alpha_i \in \mathbb{Z}_p, (i = 0, \ldots, q-1)$ from $s_i, (i = 1, \ldots, q-1)$. Algorithm $\mathcal{B}_1$ computes:
   $$g_1' \leftarrow \prod_{i=0}^{q-1} A_i^{\alpha_i} = g_1^{f(x)} \in \mathbb{G}_1$$
   Algorithm $\mathcal{B}_1$ also computes $z' = e(g_1', g_2)$ and $v' = z'^y$. The public key of the signature scheme given $\mathcal{A}_1$ is (g_1', g_2, h, v', z').
   (b) **Simulation signing oracle:** Upon receiving a query to the signing oracle, algorithm $\mathcal{B}_\infty$ simulates the reply to $\mathcal{A}_1$ as follows:
   Let $f_i(X) \leftarrow f(X)/(X + s_i) = \prod_{j=1, j\neq i}^{q-1}(X + s_i) = \sum_{j=1}^{q-2} \beta_j X^j$. For each query i ($i = 1, \ldots q-1$) with message m_i from $\mathcal{A}_1$ to the signing oracle, algorithm $\mathcal{B}_1$ selects $r_i \in Z_p$, and computes:
   $$\sigma_i \leftarrow \left(\prod_{j=0}^{q-2} (A_j)^{\beta_j}\right)^{r_i + m_i y} = \left(g_1^{f_i(x)}\right)^{r_i + m_i y} = \left(g_1^{f(x)/(x+s_i)}\right)^{r_i + m_i y}$$
   $$= (g_1')^{(r_i + m_i y)/(x + s_i)}$$
   $\mathcal{B}_1$ returns (σ_i, r_i, s_i) to $\mathcal{A}_1$ as the reply to the query. Clearly, this is a valid signature for public-key (g_1', g_2, h, v', z') and the distribution is exactly the same as that given by the signing oracle.
   (c) **Output:** Eventually, assume the forger $\mathcal{A}_1$ outputs a forgery $(m_*, \sigma_*, r_*, s_*)$ such that (σ_*, r_*, s_*) is a valid forgery distinct from any previously given signature on message m_*. The signature σ_* should satisfy: $\sigma_* = (g')_1^{(r_* + m_* y)/(x + s_*)}$, since the forgery is valid, i.e.:
   $$e(\sigma_*, h \cdot g_2^{s_*}) = e(\sigma_*, g_2^x \cdot g_2^{s_*}) = (z')^{r_*} \cdot (v')^{m_*}$$
   Let $f = (r_* + m_* y)$, $c(X) \leftarrow \sum_{i=0}^{q-2} \omega_i X^i$ and $d \in \mathbb{Z}_p$ such that $f(X) \equiv c(X)(X + s_*) + d \pmod p$. Algorithm $\mathcal{B}_1$ computes:
   $$\eta \leftarrow \left(\sigma_* \prod_{i=0}^{q-2} A_i^{-f\omega_i}\right)^{1/d} = \left((g_1^{f(x)})^{f/(x+s_*)} g_1^{-f \cdot c(x)}\right)^{1/d}$$
   $$= \left(g_1^{(c(x)(x+s_*)+d) \cdot f/(x+s_*) - f \cdot c(x)}\right)^{1/d} = \left(g_1^{f \cdot c(x) + fd/(x+s_*) - f \cdot c(x)}\right)^{1/d}$$
   $$= g_1^{f/(x+s_*)}$$

Hence, $\eta^{x+s_*} = g_1^f$.

Algorithm $\mathcal{B}_1$ outputs (η, f, s_*) as the solution to the submitted instance of the SDH2 problem.

4. **Type-2 forger.** ($c_{type} = 2$) Second, we describe the simulator $\mathcal{B}_2$, which interacts with a type-2 forger $\mathcal{A}_2$ to produce a forged signature as follows:

   (a) **Key setup:** Algorithm $\mathcal{B}_2$ randomly selects $a, b, s_i (i = 1, \ldots, q-1) \in \mathbb{Z}_p$, and randomly selects $k \in \{1, \ldots, q-1\}$.

   As stated above, let $f(X) \leftarrow \prod_{i=1}^{q-1}(X + s_i) = \sum_{i=0}^{q-1} \alpha_i X^i$, $f_i(X) \leftarrow f(X)/(X + s_i) = \prod_{j=1, j\neq i}^{q-1}(X+s_i) = \sum_{j=1}^{q-2} \beta_j^{(i)} X^j$ and $f_{k,i}(X) \leftarrow f(X)/((X+s_k)(X+s_i)) = \prod_{j=1, j\neq k,i}^{q-1}(X+s_i) = \sum_{j=1}^{q-3} \gamma_j X^j$. Algorithm $\mathcal{B}_2$ computes:

$$g_1' \leftarrow \prod_{i=0}^{q-2} A_i^{\beta_i^{(k)}} = g_1^{f_k(x)} \in \mathbb{G}_1, \qquad z' = e(g_1', g_2)$$

$$v' = e((\prod_{i=0}^{q-1} A_i^{\alpha_i})^a (\prod_{i=0}^{q-2} A_i^{\beta_i^{(k)}})^b, g_2) = e(g_1^{af(x)-bf_k(x)}, g_2) = (z')^{a(x+s_k)-b} = (z')^y.$$

   The public key of the signature scheme given $\mathcal{A}_2$ is (g_1', g_2, h, v', z').

   (b) **Simulation signing oracle:** Upon receiving a query to the signing oracle, algorithm $\mathcal{B}_2$ simulates the reply to $\mathcal{A}_2$ as follows:

   For each query i $(i = 1, \ldots, k-1, k+1, \ldots, q-1)$ with message m_i from $\mathcal{A}_2$ to the signing oracle, algorithm $\mathcal{B}_2$ selects $r_i \in Z_p$, and computes:

$$\begin{aligned}
\sigma_i &\leftarrow \left(\prod_{j=0}^{q-3} (A_j)^{\gamma_j}\right)^{r_i} \left(\prod_{j=0}^{q-2} A_j^{\beta_j^{(i)}}\right)^{m_i a} \left(\prod_{j=0}^{q-3} A_j^{\gamma_j}\right)^{-m_i b} \\
&= \left(g_1^{f_{k,i}(x)}\right)^{r_i} \left(g_1^{(af_i(x)-bf_{k,i}(x))}\right)^{m_i} = (g_1')^{r_i/(x+s_i)} \left((g_1')^{a(x+s_k)/(x+s_i)} (g_1')^{-b/(x+s_i)}\right)^{m_i} \\
&= (g_1')^{(r_i+(a(x+s_k)-b)m_i)/(x+s_i)} = (g_1')^{(r_i+m_i y)/(x+s_i)}.
\end{aligned}$$

   $\mathcal{B}_2$ returns (σ_i, r_i, s_i) to $\mathcal{A}_2$ as the reply to the query. Clearly, this is a valid signature for public key (g_1', g_2, h, v', z').

   For the query $k-th$ with message m_k from $\mathcal{A}_2$ to the signing oracle, algorithm $\mathcal{B}_2$ computes $r_k \leftarrow m_k b$, and

$$\begin{aligned}
\sigma_k &\leftarrow \left(\prod_{j=0}^{q-2} (A_j)^{\beta_j^{(k)}}\right)^{m_k a} = g_1^{m_k a f_k(x)} = \left(g_1^{r_k f_k(x)} g_1^{m_k a f(x)} g_1^{-m_k b f_k(x)}\right)^{1/(x+s_k)} \\
&= \left((g_1')^{r_k} (g_1')^{m_k(a(x+s_k)-b)}\right)^{1/(x+s_k)} = (g_1')^{(r_k+m_k y)/(x+s_k)}.
\end{aligned}$$

   $\mathcal{B}_2$ returns (σ_k, r_k, s_k) to $\mathcal{A}_2$ as the reply to the query. Clearly, this is a valid signature for public-key (g_1', g_2, h, v', z').

   (c) **Output:** Eventually, assume the forger $\mathcal{A}_2$ outputs a valid forgery $(m_*, \sigma_*, r_*, s_*)$, $\mathcal{B}_2$ checks whether $s_* = s_k$ and $r_* + m_* y \neq r_k + m_k y$. Let $f = r_*/m_* - r_k/m_k$, $h(X) \leftarrow \sum_{i=0}^{q-3} \omega_i X^i$ and $e \in \mathbb{Z}_p$ such that $f_k(X) \equiv h(X)(X + s_k) + e \pmod p$. Algorithm $\mathcal{B}_2$ computes:

$$\eta \leftarrow \left((\sigma_*^{1/m_*}/\sigma_k^{1/m_k}) \prod_{i=0}^{q-3} A_i^{-f\omega_i} \right)^{1/e} = \left((g_1')^{f/(x+s_k)} g_1^{-fh(x)} \right)^{1/e}$$
$$= \left(g_1^{f_k(x)f/(x+s_k)} g_1^{fh(x)} \right)^{1/e} = g_1^{(f_k(x)/(x+s_k)-h(x))f/e}$$
$$= g_1^{(e/(x+s_k)+h(x)-h(x))f/e} = g_1^{f/(x+s_k)}$$

Hence, $\eta^{x+s_k} = g_1^f$.
Algorithm $\mathcal{B}_2$ outputs (η, f, s_k) as the solution to the submitted instance of the SDH2 problem.

5. **Type-3 forger.** ($c_{type} = 3$) Finally, we describe the simulator $\mathcal{B}_3$, which interacts with a type-3 forger $\mathcal{A}_3$ to produce a forged signature as follows:

   (a) **Key setup:** Algorithm $\mathcal{B}_3$ randomly selects $x' \in \mathbb{Z}_p$. Algorithm $\mathcal{B}_3$ computes:
   $g_1' \leftarrow A_0 = g_1, \quad g_2' \leftarrow g_2, \quad u' = (g_2')^{x'}, \quad z' = e(g_1', g_2'), \quad v' = e(g_1, A_1) = z'^x = z'^{y'}$
   The public key of the signature scheme given $\mathcal{A}_3$ is (g_1', g_2', u', v', z').

   (b) **Simulation signing oracle:** Since $\mathcal{B}_3$ knows x', the simulation of the signing oracle exactly replicates the signing oracle.

   (c) **Output:** Eventually, assume the forger $\mathcal{A}_3$ outputs a valid forgery $(m_*, \sigma_*, r_*, s_*)$, $\mathcal{B}_3$ checks whether $s_* \in \{s_1, \ldots s_{q_s}\}$ (i.e., $s_* = s_k, k \in \{1, \ldots, q_s\}$) and $r_* \neq r_k$. If $s_* \notin \{s_1, \ldots s_{q_s}\}$ or $r_* = r_k$, then $\mathcal{B}_3$ outputs failure and aborts. Otherwise, $\mathcal{B}_3$ computes:

   $$y_* \leftarrow (r_k - r_*)/(m_k - m_*)$$

   and checks whether $A_1 = A_0^{y_*}$. If it holds, then $y_* = y' = x$. Algorithm $\mathcal{B}_3$ then randomly selects $c \in \mathbb{Z}_p \backslash \{-y_*\}$ and computes:

   $$\eta = g_1^{f/(y_*+c)} = g_1^{f/(x+c)},$$

   where $f = r_* + m_* y_*$. Hence, $\eta^{x+c} = g_1^f$.
   Algorithm $\mathcal{B}_3$ outputs (η, f, c) as the solution to the submitted instance of the SDH2 problem.

We then obtain the probability ϵ' that $\mathcal{B}$ breaks the q-SDH2 assumption as follows:

- If Type-1 forgery occurs, $\mathcal{B}$ does not abort, hence it breaks the q-SDH2 assumption.
- If Type-2 forgery occurs, $\mathcal{B}$ does not abort (breaks the q-SDH2 assumption) with probability $1/q_s$.
- If Type-3 forgery occurs, $\mathcal{B}$ does not abort, hence it breaks the q-SDH2 assumption.

Since c_{type} is independent of the type of forgery, $\mathcal{B}$ breaks the q-SDH2 assumption with probability at least $\epsilon/(3q_s)$ □

3.5 Variant of the Proposed Signature Scheme

In this section, we present a modified signature scheme from the above signature scheme (Section 3.1). The goal is to give a modified signature that can easily be extended to a new multignature scheme (see section 4).

We also use the setup parameters as in the signature scheme in Section 3.1. The modified signature scheme, called $\mathcal{BS}$, is described as follows:

- $\mathcal{BS}.\mathcal{K}$. (Key generation phase): Select random generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$, and random integers $x, y \in \mathbb{Z}_p^{\times}$. Compute $z = e(g_1, g_2) \in \mathbb{G}_T$ and $u = z^y \in \mathbb{G}_T$.The public key is the quadruple (g_1, g_2, u, z). The secret key is the pair (x, y).

- $\mathcal{BS}.\mathcal{S}$. (Signing phase): Given a secret key (x, y) and a message $m \in \mathbb{Z}_p$, pick randomly $r \in \mathbb{Z}_p \setminus \{-my\}$, $s \in \mathbb{Z}_p \setminus \{-x\}$ and compute $v = g_1^{\frac{r+my}{x+s}}$, $\sigma_1 = v^x \in \mathbb{G}_1$, $\sigma_2 = v^s \in \mathbb{G}_1$. The signature is the triple (σ_1, σ_2, r).

  Here $\frac{r+my}{x+s}$ is computed modulo p.

- $\mathcal{BS}.\mathcal{V}$. (Verification phase): Given a public key (g_1, g_2, u, z), a message m, and a signature (σ_1, σ_2, r), verify that $(g_1, g_2, \sigma_1, \sigma_2, u^m, z^r)$ is a DDH (*Decisional Diffie-Hellman*) tuple by testing whether

$$e(\sigma_1, g_2) \cdot e(\sigma_2, g_2) = u^m \cdot z^r. \tag{2}$$

  If the equality holds then the signature is declared valid; otherwise it is declared invalid.

Theorem 3.3 *If Eq.* (2) *holds, then the signature of message m is verified.*

Proof We need to show that for all key pairs and all messages, any signature generated by the signing procedure verifies as valid under the corresponding public key. Indeed, we have

$$\begin{aligned} e(\sigma_1, g_2) \cdot e(\sigma_2, g_2) &= e(v^x, g_2) \cdot e(v^s, g_2) = e(v, g_2^{x+s}) = e(g_1^{\frac{r+my}{x+s}}, g_2^{x+s}) \\ &= e(g_1, g_2)^{(x+s)\cdot\frac{r+my}{x+s}} = e(g_1, g_2)^{my} \cdot e(g_1, g_2)^r = u^m \cdot z^r. \end{aligned}$$

Theorem 3.4 *If the proposed signature scheme of Section 3.1 is secure then the modified signature scheme is secure.*

It is easy to show that the modified signature scheme is secure if the proposed signature scheme of Section 3.1 is secure. In fact, an adversary who trying to break σ_2 in the modified scheme shall face the same difficulty as if trying to break the signature σ in the proposed scheme of Section 3.1. To break σ_1 in the modified scheme (section 3.5), he may even face more difficulty because he does not know x as the secret key. Hence, we can conclude that the modified signature scheme of Section 3.5 is secure if the proposed signature scheme of Section 3.1 is secure. □

Below, we present a new non-interactive multisignature based on the modified signature scheme, which is provably secure without random oracles.

4 Multisignature

In this section, we describe the multisignature scheme $\mathcal{MBS} = (\mathcal{MK}, \mathcal{MS}, \mathcal{MV})$ extended from the $\mathcal{BS}$ above signature scheme.

Let $S = \{S_1, S_2, \ldots, S_n\}$ be a group of n users. Let I be the global information string. Each user S_i runs the randomized *key generation* algorithm that takes the global information I and outputs a triple (x_k, y_k, u_k). A possibly randomized *multisignature generation* algorithm is an interactive protocol run by an arbitrary subset of users $L \subseteq S$. The input of each user $S_i \in L$ is a message $m \in \mathbb{Z}_p$, the global information I and the pair of secret keys (x_i, y_i). The output of the algorithm is a tuple $T = (m, L, \sigma_1, \sigma_2, r)$. A deterministic *verification* algorithm takes the public keys of all users in L and T and outputs valid or invalid.

4.1 Proposed Scheme

Let $\mathbb{G}_1, \mathbb{G}_2$ be a bilinear groups pair as shown in Section 2.2 where $|\mathbb{G}_1| = |\mathbb{G}_2| = p$ for some prime p. I is the global information that consists of g_1, g_2 and $z = e(g_1, g_2)$. Let $S = \{S_1, S_2, \ldots, S_n\}$ be a group of signers.

- $\mathcal{MBS}.\mathcal{MK}$. (Key generation): Select random generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$ and random integers $x_i, y_i \in \mathbb{Z}_p^\times$ for user $S_i, 1 \leq i \leq n$. Compute $u_i = z^{y_i} \in \mathbb{G}_T$.

  The public key is u_i and the secret key is the pair (x_i, y_i) for user $S_i, 1 \leq i \leq n$.

- $\mathcal{MBS}.\mathcal{MS}$. (Multisignature creation): Any user $S_i \in S$ with secret key (x_i, y_i) that wishes to sign a message $m \in \mathbb{Z}_p$, picks a random pair $r_i \in \mathbb{Z}_p \backslash \{-my_i\}$, $s_i \in \mathbb{Z}_p \backslash \{-x_i\}$, and computes $v_i = g_1^{\frac{r_i+my_i}{x_i+s_i}} \in \mathbb{G}_1$, $\sigma_{1i} = v_i^{x_i} \in \mathbb{G}_1$, and $\sigma_{2i} = v_i^{s_i} \in \mathbb{G}_2$. Here $\frac{r_i+my_i}{x_i+s_i}$ is computed modulo p.

  The signature of user S_i is the triple $(\sigma_{1i}, \sigma_{2i}, r_i)$. S_i sends it to a designated user D (which can be implemented by any user). Let $L = \{S_{i_1}, S_{i_2}, \ldots, S_{i_l} \in S\}$ be a subgroup of users contributing to the signing. After getting all the $(\sigma_{1j}, \sigma_{2j}, r_i)$ for $j \in J = \{i_1, i_2, \ldots, i_l\}$, D computes the multisignature $(\sigma_1 = \prod_{j\in J} \sigma_{1j})$, $(\sigma_2 = \prod_{j\in J} \sigma_{2j})$, $r = \sum_{j\in J} r_i$ and outputs $T = (m, L, \sigma_1, \sigma_2, r)$.

- $\mathcal{MBS}.\mathcal{MV}$. (Multisignature verification): Given $T = (m, L, \sigma_1, \sigma_2, r)$ and the list of public keys of users in $L : u_j = z^{x_j}, j \in J = \{i_1, \ldots, i_l\}$, the verifier computes $u = \prod_{j\in J} u_j$ and verifies.

$$e(\sigma_1, g_2) \cdot e(\sigma_2, g_2) = u^m \cdot z^r. \tag{3}$$

4.2 Correctness of the scheme

Theorem 4.1 *If Eq.* (3) *holds, then the multisignature of message m is verified.*

Proof We use the method of mathematical induction to prove this theorem. First, we check if Eq. 3 is true for only one user. Assume S_1 is that user. We have:

$$e(\sigma_{11}, g_2) \cdot e(\sigma_{21}, g_1) = u_1^m \cdot z^{r_1} \tag{4}$$

Consider the left-hand side of Eq. (4):

$$\begin{aligned} e(\sigma_{11}, g_2) \cdot e(\sigma_{21}, g_1) &= e(v_1^{x_1}, g_2) \cdot e(v_1^{s_1}, g_2) \\ &= e(g_1^{\frac{r_1+my_1}{x_1+s_1}}, g_2)^{x_1+s_1} = e(g_1, g_2)^{r_1+my_1}. \end{aligned} \tag{5}$$

Consider the right-hand side of Eq. (4):

$$u_1^m \cdot z^{r_1} = e(g_1, g_2)^{my_1} \cdot e(g_1, g_2)^{r_1} = e(g_1, g_2)^{r_1+my_1}. \tag{6}$$

From Eq. (5) and Eq. (6), Eq. (4) holds. Thus Eq. (3) holds for one user.

Now we have to show that if Eq. 3 holds for $k-1, k \in [1, l-1]$ users, then it also holds for k users.

$$e(\prod_{i=1}^{k} \sigma_{1i}, g_2) \cdot e(\prod_{i=1}^{k} \sigma_{2i}, g_1) = (\prod_{i=1}^{k} u_i)^m \cdot z^{\sum_{i=1}^{k} r_i} \tag{7}$$

Assume Eq. (3) holds for $k-1$ users, i.e.,

$$e(\prod_{i=1}^{k-1} \sigma_{1i}, g_2) \cdot e(\prod_{i=1}^{k-1} \sigma_{2i}, g_1) = (\prod_{i=1}^{k-1} u_i)^m \cdot z^{\sum_{i=1}^{k-1} r_i} \tag{8}$$

$$\text{Eq. (7)} \quad \Leftrightarrow \quad e(\prod_{i=1}^{k-1} \sigma_{1i} \cdot \sigma_{1k}, g_2) \cdot e(\prod_{i=1}^{k-1} \sigma_{2i} \cdot \sigma_{2k}, g_1) = (\prod_{i=1}^{k-1} u_i)^m \cdot u_k^m \cdot z^{\sum_{i=1}^{k-1} r_i} \cdot z^{r_k} \tag{9}$$

Combine Eq. (8) with Eq. (9):

$$e(\sigma_{1k}, g_2) \cdot e(\sigma_{2k}, g_2) = u_k^m \cdot z^{r_k} \tag{10}$$

Eq. (10) is similar to Eq. (4). Thus Eq. (7) holds.

Consequently, we can conclude that Eq. (3) holds for l user. Theorem 4.1 is proved. □

4.3 Performance

In this section, we compare the scheme we have presented to the Boldyreva scheme [Bol04]. We analyze the efficiency of our scheme in relation to three indicators: signature size, complexity and security.

Bandwidth. A signature contains elements $(m, L, \sigma_1, \sigma_2, r)$, where L is the list of user, $\sigma_1 \in \mathbb{G}_1$, $\sigma_2 \in \mathbb{G}_2$ and $r \in \mathbb{Z}_p$, the last three elements, each of length is approximately $log_2 p$ bits. Compared to the multisignature of Boldyreva, ours adds two elements (σ_2, r). Their length is approximately $2log_2 p$ bits (for example, if we use an elliptic curve described in [BLS01], then their length is approximately 340 bits) which is insignificant compared to L. Therefore, we obtain a signature whose length is approximately the same as a Boldyreva-signature with the same security.

Complexity. Key generation and multisignature verification times of our scheme are comparable to Boldyreva scheme. Multisignature creation time of our scheme is much faster than the Boldyreva scheme because we do not use the expensive special hash function *Map-to-point*. Our signing algorithm makes two exponentiation to the fixed base g_1, g_2, and this can be greatly accelerated with a moderate amount of reusable pre-computation.

Security. The following theorem 4.2 states that the proposed multisignature is existentially unforgeable in the strong sense under an adaptive chosen message attack. Based on the fact that SDH2 assumption holds in $(\mathbb{G}_1, \mathbb{G}_2)$, our scheme is provably secure in the standard model whereas the Boldyreva multisignature is secure in the random oracle model.

4.4 Security

The attacks of an adversary against multisignature schemes are to forge a subgroup L and a multisignature of some message such that the latter is accepted by a verifier whereas some users of the subgroup L did not sign the message. Micali et al. [MOR01] gave the first formal notion of security for multisignature, called *ASM* model. In this section, we prove the security of our scheme in a variant of *ASM* model proposed by Boldyreva [Bol04]. In this model, the adversary $\mathcal{A}$ is given the global information I, a randomly generated public key u_{s_1} corresponding to a single honest user (referred to as S_1), and a signing oracle for that key. His goal is to output a forged message-subgroup-multisignature tuple $(M, L, \sigma_1, \sigma_2, r)$, such that $S_1 \in L, \mathcal{MBS}.\mathcal{MV}(M, L, \sigma_1, \sigma_2, r) = 1$ and S_1 did not complete the multisignature generation protocol on the input message M. In this model, we also make the *knowledge of secret key* (KOSK) assumption to prove the security of our scheme against well-known *rogue-key attacks*.

Notice that the adversary (or adversaries) that would plot the above attacks might be an insider or outsider of the group.

Theorem 4.2 *Suppose the $\mathcal{BS}$ signature scheme is (t', q, ϵ)-secure. Then, the $\mathcal{MBS}$ multisignature scheme is also (t, q, ϵ)-secure against strong existential forgery under a chosen message attack in the standard model provided that*

$$t' \geq t + \Theta(qT)$$

where T is the maximum time for an exponentiation in $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{Z}_p$.

In the following section, it should be noticed that $\mathsf{Mult\ Adv}_{MBS,I}(\mathcal{A})$ denotes the probability the forger $\mathcal{A}$ break successfully a multisignature scheme.

Proof Assume $\mathcal{A}$ is a forger that $(q, t, \epsilon)-$breaks the multisignature scheme $\mathcal{MBS}$. We construct an algorithm $\mathcal{B}$ which, by interacting with the forger $\mathcal{A}$, $(q, t', \epsilon)-$breaks the $\mathcal{BS}$ signature scheme. We prove that $\mathsf{Mult\ Adv}_{MBS,I}(\mathcal{A}) = \mathsf{Sig\ Adv}_{BS,I}(\mathcal{B}) = \epsilon$. This mean that if $\mathcal{BS}$ is a secure signature scheme then $\mathcal{MBS}$ is a secure multisignature scheme.

Algorithm $\mathcal{B}$ is given a public key $u_1 = e(g_1, g_2)^{y_1}$ of a challenger S_1 and the global information I that consists of (g_1, g_2, z). The objective of $\mathcal{B}$ is to produce a triple (σ_1, σ_2, r) for some message $M \in \mathbb{Z}_p$. Algorithm $\mathcal{B}$ does so by interacting with the forger $\mathcal{A}$ as follows:

- **Setup:** Algorithm $\mathcal{B}$ selects a list of q random distinct messages $m_1, \ldots, m_q \in \mathbb{Z}_p$, which it sends to the challenger S_1. The challenger responds with a valid public key (g_1, g_2, u_1) and a list of q signature triples $(\sigma_{11}, \sigma_{21}, r_1), \ldots, (\sigma_{1q}, \sigma_{2q}, r_q)$. To proceed, simulator $\mathcal{B}$ invokes $\mathcal{A}$, providing $\mathcal{A}$ with the public key of S_1, $PK_{S_1} = u_1$

- **Signature queries:** The attacker $\mathcal{A}$ adaptively asks for signatures on the list of message m_i. In order to respond, $\mathcal{B}$ requests a signature on m_i in turn from its own signing oracle, and returns the result to the adversary.

- **Output:** Eventually, assume the forger $\mathcal{A}$ returns a forgery $(L, \sigma_{1*}, \sigma_{2*}, r_*)$ on some message m_*, along with the set $\{r_i, 1 \leq i \leq l\}$. In addition, it outputs the set of $(l-1)$ public and secret key triples $(z^{y_2}, x_2, y_2), (z^{y_3}, x_3, y_3), \ldots, (z^{y_{l-1}}, x_{l-1}, y_{l-1})$ for all keys except the challenge key. Since the forgery is valid, we have:

$$e(\sigma_{1*}, g_2) \cdot e(\sigma_{2*}, g_2) = u^{m_*} \cdot z^{r_*},$$

where $u = \prod_{i=1}^{l} u_i = \prod_{i=1}^{l} z^{y_i},\ r_* = \sum_{i=1}^{l} r_i$. $\mathcal{B}$ computes $\sigma_1 = \sigma_{1*} \cdot \prod_{j \in J/\{1\}} g_1^{-(r_j + m_* y_j)}$ and outputs $(m_*, \sigma_1, \sigma_{2*}, r_1)$. Then, we have:

$$\begin{aligned} e(\sigma_1, g_2) \cdot e(\sigma_{2*}, g_2) &= e(\sigma_{1*}, g_2) \cdot e(\sigma_{2*}, g_2) \cdot e(\prod_{j \in J/\{1\}} g_1^{-(r_j + m_* y_j)}, g_2) \\ &= u^{m_*} \cdot z^{r_*} \cdot e(g_1, g_2)^{-\sum_{j \in J/\{1\}} (r_j + m_* y_j)} \\ &= e(g_1, g_2)^{\sum_{j \in J} (r_j + m_* y_j)} \cdot e(g_1, g_2)^{-\sum_{j \in J/\{1\}} (r_j + m_* y_j)} \\ &= e(g_1, g_2)^{r_1 + m_* y_1} = u_1^{m_*} \cdot z^{r_1}. \end{aligned}$$

So $(\sigma_1, \sigma_{2*}, r_1)$ is a valid BS signature on m_*.

Thus $\mathcal{B}$ succeeds in forgery whenever $\mathcal{A}$ is successful. Algorithm $\mathcal{B}$ makes exactly as many signing queries as $\mathcal{A}$ does. Its running time is comparable to the running time of $\mathcal{A}$, plus the time required for setup and output. □

5 Conclusions

In this paper we proposed a new signature scheme and we extended it for a new multisignature scheme. Our constructions are based on a variant of *Strong Diffie-Hellman assumption*, SDH2 assumption, and thus they are provably secure without random oracles. We compared our schemes with other works and showed that our proposed signature scheme is more efficient than that of Okamoto in [Oka06] which is based on a variant of SDH assumption. Our multisignature which is provably secure in standard model is more efficient and easier to implement than that of Boldyreva in [Bol04] due to the fact that we do not need the expensive special hash function *Map-to-point*.

Another advantage of our modified signature scheme in Section 3.5 is that it support batch verification. It can easily be extended to a new aggregate signatures, which like a special case of multisignatures for signing different messages, in standard model.

Acknowledgments

We would like to thank Alexis Bonnecaze and reviewers of SARSSI2007's conference for their helpful comments on this paper.

References

[ADR02] Jee Hea An, Yevgeniy Dodis, and Tal Rabin. On the security of joint signature and encryption. In *EUROCRYPT '02: Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques*, pages 83–107, London, UK, 2002. Springer-Verlag.

[BB00] Dan Boneh and Xavier Boyen. Short signatures without random oracles and the sdh assumption in bilinear groups. *Journal of Cryptology*, ??(??):???–???, 200? Available at `http://www.cs.stanford.edu/~xb/joc07/`.

[BB04a] D. Boneh and X. Boyen. Short signatures without random oracles. In *Advances in Cryptology—EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 56–73. Berlin: Springer-Verlag, 2004.

[BB04b] Dan Boneh and Xavier Boyen. Efficient selective-ID secure identity based encryption without random oracles. In *Advances in Cryptology—EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 223–238. Berlin: Springer-Verlag, 2004. Available at `http://www.cs.stanford.edu/~xb/eurocrypt04b/`.

[BF01] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In *CRYPTO '01: Proceedings of the 21st Annual International Cryptology Conference on Advances in Cryptology*, pages 213–229, London, UK, 2001. Springer-Verlag.

[BGHS07] Paulo S. Barreto, Steven D. Galbraith, Colm O Heigeartaigh, and Michael Scott. Efficient pairing computation on supersingular abelian varieties. *Des. Codes Cryptography*, 42(3):239–271, 2007.

[BGLS03] D. Boneh, C. Gentry, B. Lynn, and H. Shacham. Aggregate and verifiably encrypted signatures from bilinear maps, 2003.

[BLS01] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *ASIACRYPT '01: Proceedings of the 7th International Conference on the Theory and Application of Cryptology and Information Security*, pages 514–532, London, UK, 2001. Springer-Verlag.

[BLS04] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. Efficient implementation of pairing-based cryptosystems. *J. Cryptol.*, 17(4):321–334, 2004.

[Bol04] Alexandra Boldyreva. Efficient threshold signature, multisignature and blind signature schemes based on the gap-diffie-hellman-group signature scheme. In *Public Key Cryptography - PKC 2003*, volume Volume 2567/2003 of *Lecture Notes in Computer Science*, pages 31–46. Springer Berlin/Heidelberg, 2004.

[Boy89] C. Boyd. Digital multisignatures. In *In: Cryptography and Coding*, pages 241–246. Oxford University Press, 1989.

[Che06] Jung Hee Cheon. Security analysis of the strong diffie-hellman problem. In Serge Vaudenay, editor, *EUROCRYPT*, volume 4004 of *Lecture Notes in Computer Science*, pages 1–11. Springer, 2006.

[DBS04] Ratna Dutta, Rana Barua, and Palash Sarkar. Pairing-based cryptographic protocols : A survey. Cryptology ePrint Archive, Report 2004/064, 2004. `http://eprint.iacr.org/`.

[FR94] G. Frey and H-G. Rück. A remark concerning m-divisibility and the discrete logarithm in the divisor class group of curves. *Mathematics of Computation*, 62:865–874, 1994.

[GMR88] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM J. Comput.*, 17(2):281–308, 1988.

[GPS06] S. Galbraith, K. Paterson, and N. Smart. Pairings for cryptographers, 2006.

[IN83] K. Itakura and K. Nakamura. A public key cryptosystem suitable for digital multisignatures. *NEC Research and Development*, 71:1–8, 1983.

[JN03] Antoine Joux and Kim Nguyen. Separating decision diffie-hellman from computational diffie-hellman in cryptographic groups. *J. Cryptology*, 16(4):239–247, 2003.

[Mil04] Victor S. Miller. The weil pairing, and its efficient calculation. *J. Cryptol.*, 17(4):235–261, 2004.

[MOR01] Silvio Micali, Kazuo Ohta, and Leonid Reyzin. Accountable-subgroup multisignatures. In *CCS '01: Proceedings of the 8th ACM conference on Computer and Communications Security*, pages 245–254, New York, NY, USA, 2001. ACM Press.

[MOV93] A. Menezes, T. Okamoto, and S. Vanstone. Reducing elliptic curve logarithms to logarithms in a finite field. In *IEEE Transactions on Information Theory*, volume 39, pages 1639–1646, 1993.

[MSK02] S. Mitsunari, R. Sakai, and M. Kasahara. A new traitor tracing. *IEICE Trans. Fundamentals*, E85-A(2):481–484, February 2002.

[Oka88] Tatsuaki Okamoto. A digital multisignature scheme using bijective public-key cryptosystems. *ACM Trans. Comput. Syst.*, 6(4):432–441, 1988.

[Oka06] Tatsuaki Okamoto. Efficient blind and partially blind signatures without random oracles. In Shai Halevi and Tal Rabin, editors, *TCC*, volume 3876 of *Lecture Notes in Computer Science*, pages 80–99. Springer, 2006.

[OO91] Kazuo Ohta and Tatsuaki Okamoto. A digital multisignature scheme based on the fiat-shamir scheme. In Hideki Imai, Ronald L. Rivest, and Tsutomu Matsumoto, editors, *ASIACRYPT*, volume 739 of *Lecture Notes in Computer Science*, pages 139–148. Springer, 1991.

[PSV06] Dan Page, Nigel Smart, and Fre Vercauteren. A comparison of mnt curves and supersingular curves. *Applicable Algebra in Engineering, Communication and Computing*, 17(5):379–392, October 2006.

[SV07] N. Smart and F. Vercauteren. On computable isomorphisms in efficient asymmetric pairing based systems. *Discrete Applied Mathematics*, 155:538–547, April 2007.

[Wei05a] Victor K. Wei. Tight reductions among strong Diffie-Hellman assumptions, 2005.

[Wei05b] Victor K. Wei. Tracing-by-linking group signatures. In Jianying Zhou, Javier Lopez, Robert H. Deng, and Feng Bao, editors, *ISC*, volume 3650 of *Lecture Notes in Computer Science*, pages 149–163. Springer, 2005.

[ZK02] Fangguo Zhang and Kwangjo Kim. Id-based blind signature and ring signature from pairings. In *ASIACRYPT '02: Proceedings of the 8th International Conference on the Theory and Application of Cryptology and Information Security*, pages 533–547, London, UK, 2002. Springer-Verlag.

[ZSS04] Fangguo Zhang, Reihaneh Safavi, and Willy Susilo. An efficient signature scheme from bilinear pairings and its applications. In *Public Key Cryptography*, volume 2947 of *Lecture Notes in Computer Science*, pages 277–290. Springer, 2004.